CSE 460: VLSI Design (Lab)

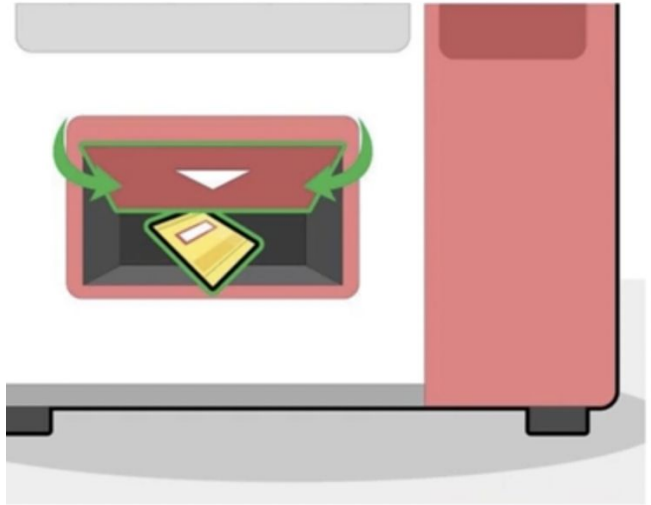
Experiment 5

Sequential Circuits Real-life Projects

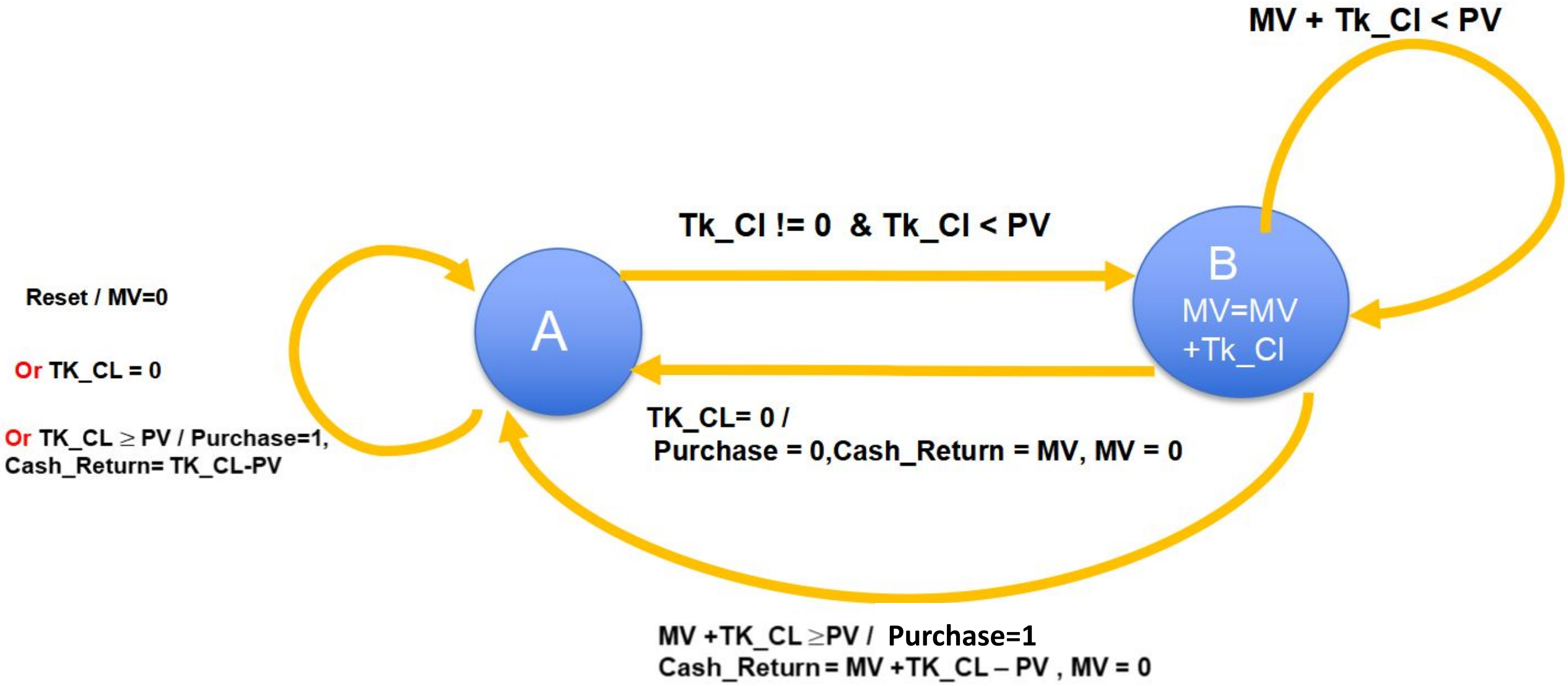
Vending Machine

What is a Vending Machine ?

What is a Vending Machine ?



Variable Product Case (State Diagram)



Description of the parameters used (IMPORTANT !!!)

PV → Maximum product value (8-bit in our example code) – This stores the binary representation of the highest product price in the vending machine.

For instance, if the machine has a 280 tk apple juice can, 150 tk muffin cake, 450 tk pineapple shake, and 200 tk chocolate: first determine the maximum product price, which is 450 tk here. This requires 9 bits in binary form. Therefore, declare [8:0] PV for this case.

Cash-in → 2-bit code for the inserted cash (00 = 0 TK, 01 = 5 TK, 10 = 10 TK, 11 = 20 TK)

If the available notes are: 0 TK (no cash inserted), 10 TK, 20 TK, 100 TK, 500 TK, and 1000 TK, then 3 bits are needed to represent these 6 note types. Hence, declare cash_in as [2:0] cash_in.

TK_CL → Stores the decimal-equivalent binary of the cash amount inserted in a given clock cycle

For example, if cash_in = 500 tk at one clock, TK_CL will store 500 (in decimal) = 111110100 (in binary); thus, 9 bits are required here.
Always find the highest note value among the available denominations and size TK_CL accordingly.
If the available notes are: 0 TK, 10 TK, 20 TK, 100 TK, 500 TK, 1000 TK, then declare [9:0] TK_CL as the largest note value is 1000 TK, which requires 10 bits to store. This 10-bit TK_CL will also cover all smaller notes' binary values.

MV → Stores the decimal-equivalent binary of the total stored money

MV is related to PV. In the state diagram, MV is updated only when prev MV + TK_CL < PV, meaning the value stored in MV is always less than PV. When MV + TK_CL >= PV, cash is returned immediately and MV is reset to zero. For safety, use the same number of bits for MV as for PV.

Cash_return → Stores the decimal-equivalent binary of the returned money when MV + TK_CL >= PV or Tk_Cl>= PV or TK_Cl==0

The maximum return amount is always less than the largest note available. For example, if the available notes are 0 TK, 10 TK, 20 TK, 50 TK, 100 TK, and you, as a buyer select the maximum price product (500 TK), inserting notes of 100 TK, 100 TK, 100 TK, 100 TK, 50 TK, 20 TK, 20 TK, and finally 100 TK will return 90 TK, which is less than the largest note (100 TK).
However, when you insert 100tk + 50 tk + 0tk for example, for a 200tk valued product, Cash_return will return total stored money ie MV.
Hence, Cash_return's size will be the same as that of MV.

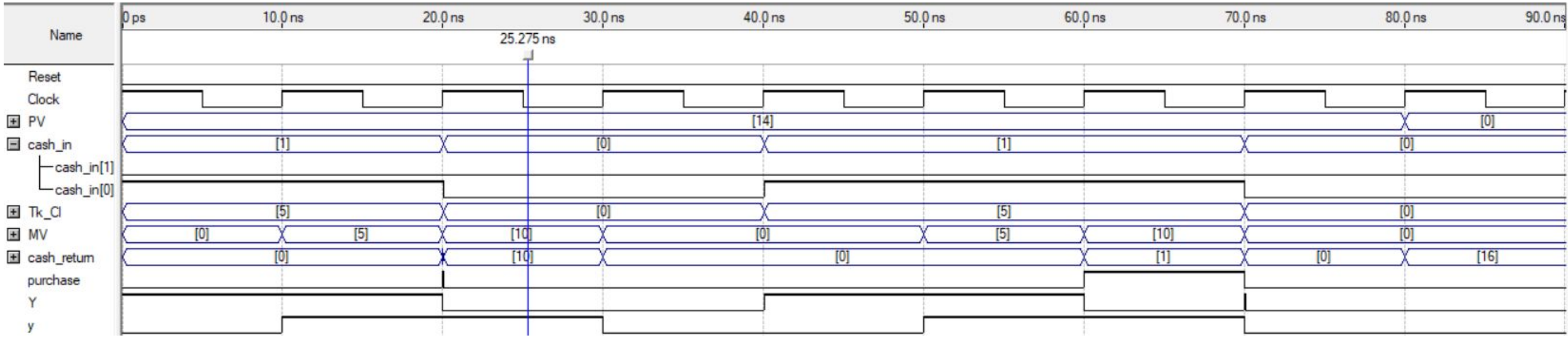
Therefore, all these parameters may vary in bit length depending on factors such as the maximum product price, total available currency denominations, etc.

```
module VM(cash_in, PV, Reset, Clock, purchase, Tk_Cl, cash_return, MV, y, Y);
/*
2 bit input to indicate cash_in=0,5,10,20 TK,
8 bit input of PV(product value)
Internal reg ,Tk_Cl= money inserted in each clock cycle(5 bit since max 20 tk)
Internal reg, MV= total money inserted in subsequent clks to buy the Product;
*/
input [1:0] cash_in;
input Reset, Clock;
input [7:0] PV;
output reg purchase;
output reg [7:0] cash_return;
output reg Y = 0;
output reg y;
output reg [7:0] MV;
output reg [4:0] Tk_Cl = 0;

parameter S0 = 0, S1 = 1;
parameter T0 = 2'b00, T5 = 2'b01, T10 = 2'b10, T20 = 2'b11;

// Next state logic and output value
always @(y, Tk_Cl, cash_in) begin
   case (cash_in)
      T5:  Tk_Cl = 5;
      T10: Tk_Cl = 10;
      T20: Tk_Cl = 20;
      T0:  Tk_Cl = 0;
   endcase

   case (y)
      S0: begin
         if (Tk_Cl >= PV) begin
            if (PV == 0) begin
               Y = S0;
               purchase = 0;
               cash_return = Tk_Cl;
            end else begin
               purchase = 1;
               cash_return = Tk_Cl - PV;
               Y = S0;
            end
         end else if (Tk_Cl == 0) begin
            purchase = 0;
            cash_return = 0;
            Y = S0;
         end else if (Tk_Cl < PV) begin
            purchase = 0;
            cash_return = 0;
            Y = S1;
         end
      end
      S1: begin
         if ((MV + Tk_Cl) >= PV) begin
            purchase = 1;
            cash_return = (MV + Tk_Cl - PV);
            Y = S0;
         end else if (Tk_Cl == 0) begin
            purchase = 0;
            cash_return = MV;
            Y = S0;
         end else if ((MV + Tk_Cl) < PV) begin
            purchase = 0;
            cash_return = 0;
            Y = S1;
         end
      end
   endcase
end

// State change with clock edge
always @(posedge Reset, posedge Clock) begin
   if (Reset == 1) begin
      y <= 0;
      MV <= 0;
   end else begin
      if (purchase | Tk_Cl == 0)
         MV <= 0;
      else
         MV <= MV + Tk_Cl;
      y <= Y;
   end
end

endmodule
```

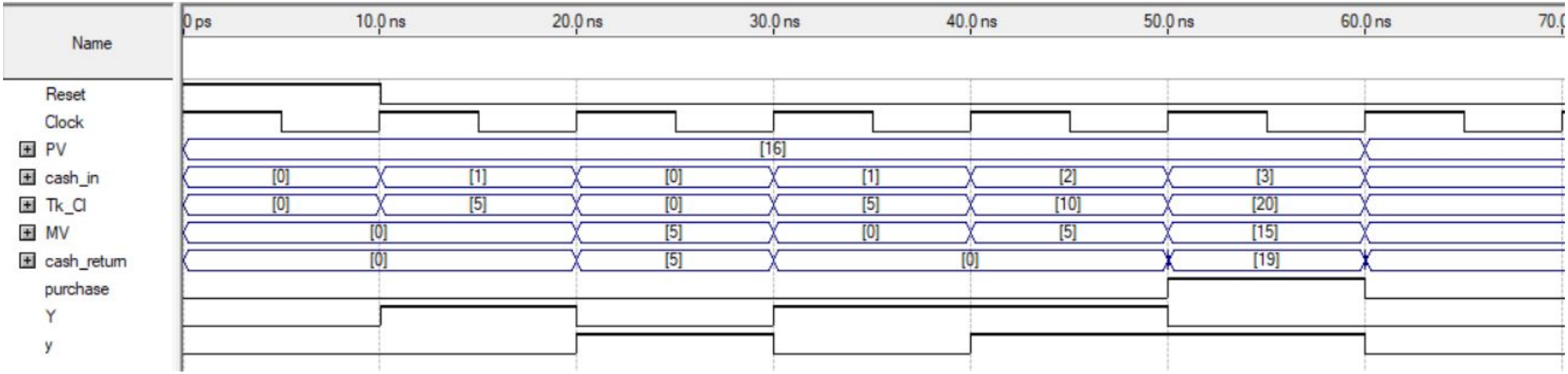
Variable Product Case (Test Case-1)

For PV=14 tk; money= 5 tk + 5 tk + 0 tk + 0 tk+ 5tk + 5tk + 5 tk

Remember the 2 steps from exp-4:
1.Invert the Clock
2. Add a little offset (10ps here) to the input waveforms cash_in and PV.



Variable Product Case (Test Case-2)

For PV=16 tk; money= 5 tk + 0 tk +5 tk + 10 tk + 20 tk

(again follow the two steps: Clock inversion and offset insertion.)